



Exam Ready

Prepare. Practice. Perform.

TECHNICAL IMPLEMENTATION PLAN

Admin Dashboard & Question Set Pipeline

End-to-end specification for the admin panel — domain model, MongoDB schemas, the teacher PDF upload format, backend API contract, frontend route map, and a phased build order.

6

New MongoDB collections

24

Admin API endpoints

11

New frontend routes

11

Delivery phases

~18

Working days estimated

1

PDF format spec for teachers

PROJECT

exam-ready

STACK

Next.js 16 · React 19 · Node · MongoDB

VERSION

Draft 1.0

DATE

19 September 2026

Scope & Assumptions

What we are building, and what we are deliberately deferring.

Build a working admin panel where an operator can create exams, and where teachers' question papers arrive as **PDF uploads** that the system parses into structured, attemptable question sets — reading the paper's duration, question count, marking scheme and section layout straight out of the file.

The problem we are solving

The platform hosts seven West Bengal exam categories. Each category has many exams, and each exam needs **many question sets** — contributed by different teachers or generated by the system. Typing 100 MCQs into a web form per set is not viable. The PDF pipeline is therefore the centre of gravity of this entire plan: everything else is conventional CRUD around it.

Working assumptions

#	ASSUMPTION	IF WRONG, IMPACT
1	Backend is Node/Express + MongoDB (Mongoose) , matching the existing <code>exam-ready-node</code> API.	Schemas become SQL tables; relationships change shape.
2	Teacher PDFs are digitally exported (Word / Google Docs → Save as PDF), not scanned photographs.	OCR becomes mandatory in Phase 4, adding ~4 days.
3	No auth or RBAC for now. The admin panel is openly reachable, but structured so auth drops back in without rewrites.	None — this is a deliberate temporary choice.
4	File storage uses the existing ImageKit <code>/media/upload</code> route already present in the frontend.	Swap the storage adapter; pipeline unchanged.

Temporary access, by design

For this phase the admin panel is entered via a plain **“Access Admin Panel”** button on the public home page. Three things are switched off and must be switched back on before launch (tracked as Phase 10):

- the `<ProtectedRoute requiredRole="admin">` wrapper in the admin layout,
- the `'/admin'` entry in the edge role-routing map,
- the home-page button itself.

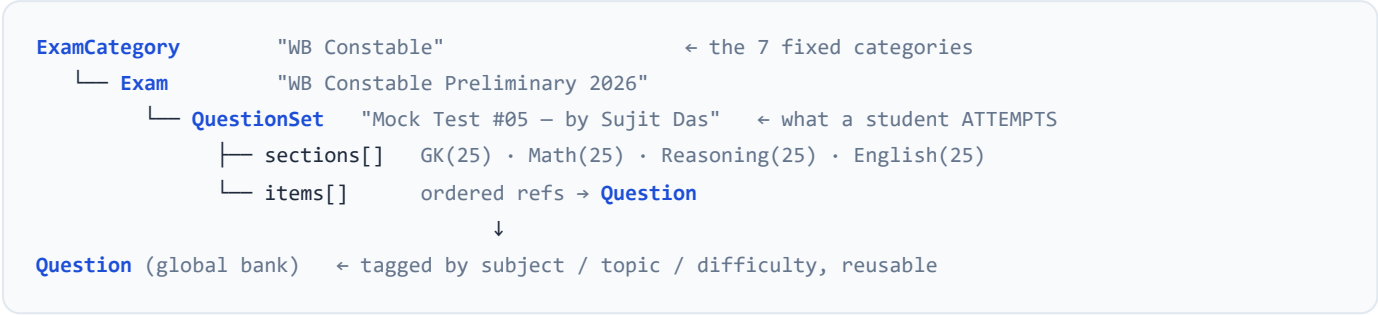
Out of scope for this phase

- Teacher-facing self-service upload portal — admins upload on their behalf for now.
- Payments, subscription gating and revenue reporting on question sets.
- OCR for scanned papers (detected and rejected with a clear message instead).
- Approval workflows / multi-stage editorial review beyond the single review screen.

The Domain Model

The single most consequential decision in this plan.

Four levels, with the **Question Set** as the unit a student actually sits:



Key design call: questions are not embedded

`Question` lives in its **own collection** — the global question bank — and `QuestionSet` holds an **ordered array of references** rather than embedded question documents.

Why references win

- A **system-generated** set references existing bank questions; embedding would force duplication.
- Daily quizzes and practice sets can draw from the same bank by subject and difficulty distribution.
- Fixing a typo in a question fixes it everywhere at once.
- The bank stays searchable and paginable independently of any paper.

The cost we accept

- Serving a paper needs a populate / join on `items.questionId`.
- Editing a shared question affects every set using it — mitigated by the `usageCount` counter and a warning in the editor.
- A per-set marks override lives on the `items` entry, not on the question.

How a set comes into existence

SOURCE	FLOW
PDF_UPLOAD	Teacher's PDF is parsed → new bank questions are created → a draft set references them in order. The primary path.
MANUAL	Admin builds a set in the editor by picking existing bank questions or authoring new ones.
SYSTEM_GENERATED	Generator picks N questions from the bank matching a subject / difficulty distribution — used for daily quizzes and practice sets.

MongoDB Collections & Schemas

Six new collections, plus alignment of the existing attempts collection.

COLLECTION	STATUS	HOLDS
examcategories	NEW	The seven fixed WB exam categories
subjects	EXTEND	Self-referencing subject / topic tree
exams	EXTEND	An exam and its default pattern
questionsets	NEW	The attemptable paper — config, sections, ordered items
questions	EXTEND	The global, reusable question bank
questionsetuploads	NEW	One PDF upload and its parse result
attempts	ALIGN	Student attempts, now keyed to a question set

examcategories

```
{
  _id,
  name: "WB Constable", slug: "wb-constable", nameBn: "ডাব্লুবি কনস্টেবল",
  description, icon, color,
  order: Number,           // display order on the public site
  isActive: Boolean,
  examCount: Number       // denormalised counter
}
// index: { slug: 1 } unique, { order: 1 }
```

subjects

```
{
  _id,
  name: "Polity", slug: "polity", nameBn: "রাষ্ট্রবিজ্ঞান",
  description, icon, color,
  parentId: ObjectId | null, // null = subject, set = topic
  questionCount: Number,
  isActive: Boolean
}
// index: { slug: 1 } unique, { parentId: 1 }
```

One self-referencing tree covers both subjects and topics, avoiding a near-duplicate `topics` collection with identical fields.

exams

```

{
  _id,
  categoryId: ObjectId → examcategories,
  title: "WB Constable Preliminary 2026",
  slug, description, instructions, // rich text shown before the paper starts
  year: Number,

  examPattern: { // DEFAULTS inherited by every new set
    duration: 60, // minutes
    totalQuestions: 100,
    marksPerQuestion: 1,
    negativeMarks: 0.25,
    passingMarks: 35,
    sections: [
      { key: "gk", name: "General Awareness", nameBn, questionCount: 25, subjectId }
    ]
  },

  language: "BILINGUAL", // EN | BN | BILINGUAL
  difficulty: "MEDIUM",
  isPaid: Boolean, price: Number,
  status: "DRAFT | PUBLISHED | ARCHIVED",
  thumbnailUrl, tags: [String],
  questionSetCount: Number, totalAttempts: Number
}
// index: { slug:1 } unique, { categoryId:1, status:1 }, { status:1, createdAt:-1 }

```

questionsets **CORE**

```

{
  _id,
  examId: ObjectId → exams,
  categoryId: ObjectId,           // denormalised for fast filtering
  title: "WB Constable Prelim – Mock Test 05",
  setNumber: Number,             // auto-increments within the exam
  description,

  source: "PDF_UPLOAD | MANUAL | SYSTEM_GENERATED",
  author: { name: "Sujit Das", id: ObjectId|null, type: "TEACHER | SYSTEM | ADMIN" },

  // ————— every field below is PARSED FROM THE PDF HEADER —————
  config: {
    duration: 60,                 totalQuestions: 100,
    totalMarks: 100,             marksPerQuestion: 1,
    negativeMarks: 0.25,         passingMarks: 35,
    shuffleQuestions: false,     shuffleOptions: false,
    allowMarkForReview: true,    showResultImmediately: true,
    showExplanations: true
  },
  language: "BN",
  difficulty: "MEDIUM",
  sections: [
    { key: "gk", name: "General Awareness", questionCount: 25, startsAt: 1, endsAt: 25 }
  ],

  items: [                       // ORDERED – this array IS the paper
    { questionId: ObjectId, questionNumber: 1, sectionKey: "gk",
      marks: 1, negativeMarks: 0.25 }
  ],

  uploadId: ObjectId → questionsetuploads, // provenance; null when manual
  status: "DRAFT | IN_REVIEW | PUBLISHED | ARCHIVED",
  validation: {
    isValid: Boolean,
    errors: [{ code, message, questionNumber }], // blocking
    warnings: [{ code, message, questionNumber }] // non-blocking
  },
  publishedAt, scheduledAt, expiresAt,
  attemptCount: Number, averageScore: Number
}
// index: { examId:1, setNumber:1 } unique, { status:1, publishedAt:-1 },
//        { categoryId:1, status:1 }

```

```

{
  _id,
  text: String,                // primary language
  textBn: String,              // bilingual pair (optional)
  imageUrl, imageUrlBn,
  type: "MCQ | TRUE_FALSE | FILL_IN_BLANK",
  options: [{ key: "A", text, textBn, imageUrl }],
  correctOptionKey: "A",       // key, NOT _id – survives re-parse and shuffling
  explanation, explanationBn,

  subjectId: ObjectId, topicId: ObjectId | null,
  difficulty: "EASY | MEDIUM | HARD",
  defaultMarks: 1, defaultNegativeMarks: 0.25,
  tags: [String],

  source: { type: "PDF_UPLOAD", uploadId, setId, pdfPage: 3, rawBlock: "..."},
  needsReview: Boolean,        // parser was not confident – flagged in review UI
  reviewNotes: [String],
  usageCount: Number,          // how many sets reference this question
  isActive: Boolean
}
// index: { subjectId:1, difficulty:1 }, { text:"text" } full-text, { needsReview:1 }

```

Why correctOptionKey and not correctOptionId

Storing the answer as the literal option key ("A") rather than a generated sub-document `_id` means re-parsing a PDF, editing an option, or shuffling options at serve time cannot silently orphan the answer. This is a change from the current frontend type, which uses `correctOptionId`.

questionsetuploads **THE PDF JOB**

```

{
  _id,
  examId: ObjectId,
  file: { url, fileId, name, sizeBytes, pageCount, mimeType },
  uploaderName: "Sujit Das",    // free text until auth exists

  status: "UPLOADED | PARSING | PARSED | NEEDS_REVIEW | COMMITTED | FAILED",

  parseResult: {
    meta: { ...raw key-value pairs from the #META block... },
    questions: [{
      number, text, options[], answerKey, subject, difficulty, explanation,
      confidence: 0.0-1.0,          // parser self-assessment
      issues: [String]
    }],
    stats: {
      declaredTotal: 100, parsedTotal: 98, missingNumbers: [47, 63],
      lowConfidence: 4, durationMs: 1840
    }
  },

  errors: [{ code, message, line, questionNumber }],
  warnings: [ ... ],
  questionSetId: ObjectId | null, // set once committed
  rawText: String,                // full extracted text, for debugging
  completedAt

}
// index: { examId:1, createdAt:-1 }, { status:1 }

```

attempts **ALIGN EXISTING**

```
{
  _id, userId, questionSetId, examId,
  startedAt, submittedAt, serverDurationSec,
  status: "ACTIVE | SUBMITTED | EXPIRED",

  answers: [{ questionId, questionNumber, selectedKey, isCorrect,
               marksAwarded, timeSpentSec, markedForReview }],

  score, totalMarks, correctCount, wrongCount, skippedCount,
  accuracy, percentile, rank,
  sectionWise: [{ sectionKey, score, correct, wrong, accuracy }]
}
// index: { userId:1, questionSetId:1 }, { questionSetId:1, score:-1 } ← Leaderboard
```

The PDF Upload Format

The contract between teachers and the parser. Share this section with contributors.

Every question paper is submitted as a PDF that begins with a **metadata block** — this is where the set's duration, question count, marking scheme and section layout come from — followed by questions in a fixed, line-anchored marker format.

Design rules

#	RULE	REASON
1	Markers start a line. Every marker (<code>Q1.</code> , <code>A</code>) , <code>ANS:</code>) appears at the beginning of its own line.	The parser anchors on line starts instead of guessing across wrapped text.
2	A metadata header is mandatory , delimited by <code>#META</code> and <code>#END_META</code> .	Removes all guesswork about timing, totals and marks.
3	Digital PDFs only. Write in Word or Google Docs, then <i>Save as PDF</i> .	Scans and photographs carry no extractable text layer.
4	No tables, columns, text boxes or images for question text.	Multi-column layouts scramble reading order during extraction.
5	Question numbers run sequentially from 1, with no gaps or repeats.	Gaps are treated as a blocking validation error.

Template

```
#META
EXAM: WB Constable Preliminary 2026
SET_TITLE: WB Constable Prelim – Mock Test 05
AUTHOR: Sujit Das
LANGUAGE: BN
DURATION: 60
TOTAL_QUESTIONS: 100
MARKS_PER_QUESTION: 1
NEGATIVE_MARKS: 0.25
PASSING_MARKS: 35
DIFFICULTY: MEDIUM
SECTION: gk | General Awareness | 25
SECTION: math | Mathematics | 25
SECTION: reasoning | Reasoning | 25
SECTION: english | English | 25
#END_META
```

Q1. সংবিধানে মৌলিক কর্তব্য অন্তর্ভুক্ত করার সুপারিশ কোন কমিটি করেছিল?

- A) স্বর্ণ সিং কমিটি
- B) সরকারি কমিশন
- C) বলবন্ত রাই মেহতা কমিটি
- D) ভার্মা কমিটি

ANS: A

SUB: Polity

DIFF: MEDIUM

EXP: ১৯৭৬ সালে স্বর্ণ সিং কমিটি মৌলিক কর্তব্য অন্তর্ভুক্তির সুপারিশ করে।

Q2. Who was the first Governor-General of Bengal under the Regulating Act of 1773?

- A) Lord Clive
- B) Warren Hastings
- C) Lord Cornwallis
- D) Lord William Bentinck

ANS: B

SUB: History

DIFF: EASY

EXP: Warren Hastings took office in 1774 under the Regulating Act of 1773.

Bilingual papers

When `LANGUAGE: BILINGUAL`, split the English and Bengali text of the same line with a triple pipe. The parser stores them as `text` and `textBn`:

Q3. What is the capital of West Bengal? ||| পশ্চিমবঙ্গের রাজধানী কোথায়?

- A) Kolkata ||| কলকাতা
- B) Howrah ||| হাওড়া
- C) Siliguri ||| শিলিগুড়ি
- D) Durgapur ||| দুর্গাপুর

ANS: A

Marker reference

MARKER	REQUIRED	NOTES
#META ... #END_META	YES	Must be the first block in the document.
EXAM:	YES	Must match an existing exam title, else the admin maps it at upload time.
SET_TITLE:	YES	Shown to students as the paper name.
DURATION:	YES	Whole minutes, digits only.
TOTAL_QUESTIONS:	YES	Cross-checked against the number actually parsed.
MARKS_PER_QUESTION:	YES	Decimals allowed.
NEGATIVE_MARKS:	NO	Defaults to 0 when absent.
PASSING_MARKS:	NO	Defaults to 33% of total marks.
LANGUAGE:	NO	EN / BN / BILINGUAL . Defaults to the exam's language.
DIFFICULTY:	NO	Set-level default for questions with no DIFF: .
SECTION:	NO	key Display Name count . Repeat one line per section. Counts must sum to TOTAL_QUESTIONS .
Q<n>.	YES	Sequential from 1. Q1. or Q1) both accepted.
A) B) C) D)	YES	Two to six options supported.
ANS:	YES	A single option key that must exist in that question.
SUB:	NO	Falls back to the section's mapped subject.
DIFF:	NO	Falls back to the set-level difficulty.
EXP:	NO	May span multiple lines; runs until the next Q marker.

Validation outcomes

Blocking errors — cannot publish

- Missing or malformed #META block
- Parsed question count ≠ TOTAL_QUESTIONS
- Gaps or duplicates in question numbering
- A question with fewer than two options
- ANS: missing, or naming an option that does not exist
- Section counts not summing to the declared total
- Fewer than 200 extractable characters → treated as a scanned PDF

Warnings — publish allowed

- Missing EXP: on one or more questions
- SUB: naming an unknown subject
- Question text under 10 characters (likely truncated)
- Duplicate question text within the set
- Parser confidence below 0.7 → flagged needsReview
- Unusually long option text (possible line-merge)

What we ship to teachers

#	ARTEFACT	PURPOSE
1	question-set-template.docx	Fill-in-the-blanks Word file with the #META block pre-written — the format is hard to break by accident.
2	question-set-sample.pdf	A complete, correctly formatted ten-question example.
3	/admin/format-guide	In-app page with the rules, side-by-side “wrong vs right” examples, and download buttons for both files above.

Backend

The parsing pipeline and the admin API contract.

The parsing pipeline

```
POST /admin/uploads/question-set (multipart, PDF)
|
├ store on ImageKit → create QuestionSetUpload { status: UPLOADED }
├ respond 202 Accepted { uploadId } ← never parse synchronously
├
└ parse job (inline for <5 MB, BullMQ queue beyond that)
  1. pdfjs-dist → per-page text items → rebuild lines by Y-coordinate
  2. if extracted characters < 200 → FAIL "scanned PDF not supported"
  3. slice the #META block → key-value map → validate with zod
  4. split the body on /^Q(\d+)[.])/m → one block per question
  5. per block: parse options, ANS, SUB, DIFF, EXP + score confidence
  6. cross-validate: count match? numbering gaps? section sums? answer key valid?
|
└ QuestionSetUpload { status: PARSED | NEEDS_REVIEW | FAILED }
|
└ admin reviews and corrects in the UI
|
POST /admin/uploads/:id/commit
└ transaction: insert Questions → create QuestionSet { status: DRAFT } → link upload
```

Libraries

- `pdfjs-dist` — text extraction with coordinates
- `zod` — metadata block validation
- `multer` — multipart upload handling
- `bullmq` — optional queue for large files

Why coordinates, not raw text

`pdf-parse` returns a flat string that merges columns and loses line breaks. Reconstructing lines from `pdfjs-dist` text items by their Y position keeps the line-anchored marker format reliable — which the whole spec depends on.

Admin API contract

METHOD	PATH	PURPOSE
GET POST	/admin/categories	Exam category list and create
PUT DEL	/admin/categories/:id	Update, delete
GET	/admin/exams?category&status&q&page	Paginated, filterable exam list
POST	/admin/exams	Create exam, including its <code>examPattern</code>
GET PUT DEL	/admin/exams/:id	Read, update, delete
PATCH	/admin/exams/:id/status	Publish / archive
GET	/admin/exams/:id/question-sets	Sets belonging to an exam
POST	/admin/uploads/question-set	PDF upload → 202 { uploadId }
GET	/admin/uploads/:id	Poll parse status and result
POST	/admin/uploads/:id/reparse	Re-run the parser after a config tweak
POST	/admin/uploads/:id/commit	Create the question set from reviewed data
GET	/admin/uploads?examId&status	Upload history
GET	/admin/question-sets?exam&status&source&q	Global set list
GET PUT DEL	/admin/question-sets/:id	Read, edit config, delete
PATCH	/admin/question-sets/:id/status	Publish — blocked while <code>validation.isValid</code> is false
POST	/admin/question-sets/:id/items	Add, remove and reorder questions
POST	/admin/question-sets/generate	System-generated set from the bank
GET POST	/admin/questions?subject&difficulty&q	Question bank browse and create
PUT DEL	/admin/questions/:id	Inline edit from the review screen
POST	/admin/questions/bulk	Bulk update, e.g. clear <code>needsReview</code>
GET POST PUT DEL	/admin/subjects	Subject / topic tree
GET	/admin/stats	Dashboard tiles — already stubbed in the frontend

Student-side endpoints

The public contract is unchanged in shape; only its data source moves to question sets.

METHOD	PATH	NOTES
GET	/exams/:id/sets	Published sets only, with attempt counts
POST	/attempts/start	Creates the attempt; server owns the clock
GET	/attempts/:id/questions	Strips <code>correctOptionKey</code> and <code>explanation</code>
POST	/attempts/:id/submit	Scored entirely server-side; returns the result

Answer-key leakage is the one bug we cannot ship

Strip `correctOptionKey` and `explanation` in a **serializer** shared by every student-facing route — not ad hoc in each controller, where one missed route exposes the whole paper. Add an integration test asserting both fields are absent from the live-attempt payload.

Frontend

Route map, the upload wizard, and the files that change.

Phase 0 — opening the door

#	CHANGE	FILE
1	Add an “Access Admin Panel” button in the hero, linking to <code>/admin</code>	<code>(public)/page.tsx</code>
2	Remove the <code><ProtectedRoute></code> wrapper, leaving a <code>TODO: re-enable before launch</code>	<code>(admin)/layout.tsx</code>
3	Comment out the <code>‘/admin’</code> entry in <code>ROLE_PREFIXES</code>	<code>proxy.ts</code>
4	Add the new sidebar entries	<code>AdminSidebar.tsx</code> , <code>config/site.ts</code>

Route map

<code>src/app/(admin)/admin/</code>	
├ <code>page.tsx</code>	existing dashboard – wire to the real <code>/admin/stats</code>
├ <code>categories/page.tsx</code>	NEW seven-category manager
├ <code>exams/</code>	
│ └ <code>page.tsx</code>	rewrite – real API, drop <code>MOCK_EXAMS_ADMIN</code>
│ └ <code>new/page.tsx</code>	NEW exam + examPattern + section builder
│ └ <code>[examId]/</code>	
│ │ └ <code>page.tsx</code>	NEW tabs: Overview Question Sets Settings
│ │ └ <code>sets/new/page.tsx</code>	NEW ★ three-step PDF upload wizard
├ <code>question-sets/</code>	
│ └ <code>page.tsx</code>	NEW global list with filters
│ └ <code>[setId]/</code>	
│ │ └ <code>page.tsx</code>	NEW set editor – config + ordered questions
│ │ └ <code>review/page.tsx</code>	NEW ★ post-parse review screen
├ <code>uploads/page.tsx</code>	NEW upload job history and status
├ <code>format-guide/page.tsx</code>	NEW ★ PDF format docs + template downloads
├ <code>questions/page.tsx</code>	rewrite – real question bank
└ <code>subjects/page.tsx</code>	rewrite – subject / topic tree

The PDF upload wizard **CRITICAL PATH**

Route: `/admin/exams/[examId]/sets/new`

STEP	WHAT THE ADMIN SEES
1. Upload	Dropzone reusing the existing <code>FileUpload</code> component — PDF only, 20 MB cap. A sidebar repeats the format rules with <i>Download Template</i> and <i>View Format Guide</i> links, so the rules are visible at the moment of upload.
2. Parsing	Polls <code>GET /admin/uploads/:id</code> every 1.5 s, showing real stages: Uploading → Extracting text → Reading metadata → Parsing questions → Validating.
3. Review	Detected configuration at the top (all editable), a validation summary, then one inline-editable card per question. Committing is blocked while blocking errors remain.

Step 3 layout

Detected from PDF

Duration 60 min

Questions 100

Marks 1

Negative 0.25

Sections: GK(25) Math(25) Reasoning(25) English(25)

✓ 96 parsed clean

⚠ 2 need review

✗ 2 errors

Q1 ✓

question text ... A) B) C) D) ANS: A [Polity] [MEDIUM]

Q47 ✗ Missing ANS

[fix inline – pick the correct option]

[Save as Draft]

[Commit & Publish]

← disabled while errors > 0

This screen decides whether the feature succeeds

No parser reaches 100% on documents written by dozens of different people. The review screen is what converts an imperfect parse into a correct paper in under two minutes — so it gets the generous share of the frontend budget, not the CRUD tables.

Files created and changed

FILE	ACTION	CONTENTS
src/types/questionSet.ts	NEW	QuestionSet , SetConfig , Section , SetItem
src/types/upload.ts	NEW	UploadJob , ParseResult , ParsedQuestion , ValidationIssue
src/types/question.ts	EDIT	Add correctOptionKey , textBn , needsReview , source
src/types/exam.ts	EDIT	Add categoryId and examPattern ; drop the flat duration/marks fields
src/lib/api/questionSets.ts	NEW	Set CRUD, items reorder, publish, generate
src/lib/api/uploads.ts	EDIT	Add uploadQuestionSetPdf , getUploadJob , commitUpload
src/lib/api/admin.ts	EDIT	Categories, exams CRUD, question bank CRUD
src/hooks/useQuestionSets.ts	NEW	React Query hooks with cache invalidation
src/hooks/useUploadJob.ts	NEW	Polling hook that stops on a terminal status
src/schemas/questionSet.schema.ts	NEW	zod schemas mirroring the backend validators
src/components/admin/*	NEW	ExamForm , SectionBuilder , PdfUploadWizard , ParsedQuestionCard , ValidationSummary , SetConfigPanel , QuestionEditor , DataTable
src/components/layout/AdminSidebar.tsx	EDIT	Categories, Question Sets, Uploads, Format Guide entries
src/config/site.ts	EDIT	Matching adminNav entries

Every admin page currently rendering `MOCK_*` constants — exams, questions, subjects, results, payments, content, users — is rewritten against the real API as its phase lands.

SECTION 06

Build Order & Estimate

Eleven phases, roughly eighteen working days.

PHASE	AREA	SCOPE	DAYS
0	Frontend	Home-page button, unlock <code>/admin</code> , sidebar navigation	0.5
1	Frontend	Types, zod schemas and API client modules — contract first, so both sides build against the same shapes	1
2	Backend	Mongoose models; Category, Exam and Subject CRUD	2
3	Frontend	Categories, Exams list / create / detail, Subjects — on real data	2
4	Backend	★ PDF parser, upload / parse / commit endpoints, validator	3
5	Frontend	★ Upload wizard and review screen	3
6	Full stack	Question bank and question set editor — reorder, swap, config edit	2
7	Content	Format guide page, <code>.docx</code> template, sample PDF	0.5
8	Backend	System-generated sets via <code>/question-sets/generate</code>	1
9	Full stack	Wire the student exam runtime to question sets; server-side scoring	2
10	Frontend	Restore auth: <code>ProtectedRoute</code> , proxy prefix, remove the home button	0.5
Σ			

Where the risk actually sits

Phases **4 and 5** — the parser and the review screen — carry almost all the uncertainty in this estimate. Everything else is conventional CRUD against well-understood shapes. If the schedule slips, it slips there, so schedule them with slack and test them against real teacher documents early rather than synthetic ones.

Suggested parallelisation

With one backend and one frontend developer, Phases 2–3 and 4–5 pair naturally: the frontend builds against the Phase 1 contract while the backend implements it. That compresses the calendar to roughly **eleven working days** without reducing the total effort.

Definition of done per phase

- No `MOCK_*` constant remains in any page the phase touches.
- Loading, empty and error states are handled — the shared UI components already exist.
- Every new endpoint is exercised at least once from the UI, not only from a REST client.
- Validation rules are enforced on the backend, not only in the form.

SECTION 07

Risks & Mitigations

What is most likely to go wrong, and what we do about it up front.

RISK	LIKELIHOOD	MITIGATION
Teachers ignore the format	High	A strict validator plus a review screen that makes fixing cheap, and a <code>.docx</code> template that is hard to break by accident. We assume non-compliance rather than hoping against it.
Scanned or photographed PDFs	High	Detect via the character-count threshold and reject with a clear, specific message. OCR stays a Phase 2 add-on that can reuse the existing <code>initiateOcr</code> stub in the admin API.
Bengali text extracted incorrectly	Medium	Test with a real Bengali paper in the first days of Phase 4, not at the end. Require Unicode-embedded fonts, and add a raw-text debug view on the upload detail page so failures are diagnosable without re-uploading.
Large papers time out	Medium	Return <code>202 Accepted</code> and poll from day one. A 200-question PDF must never be parsed inside the request cycle.
Answer keys leak to students	Low / severe	Strip in a shared serializer rather than per controller, and assert absence in an integration test. The cost of this bug reaching production is a compromised paper.
Editing a shared question silently changes published papers	Medium	Surface <code>usageCount</code> in the editor and warn before saving a question used by more than one set.
Open admin panel reachable in production	Low / severe	Phase 10 is a release blocker, not a nice-to-have. Keep all three switches in one commit so re-enabling is a single revert.

Open questions for the team

1. Should a question set be independently priced, or does it inherit the parent exam's `isPaid` and `price` ?
2. Do teachers eventually upload directly, and if so, does their work need an approval step before publication?
3. Should the same PDF uploaded twice be detected as a duplicate, by file hash or by question-text similarity?
4. Do we need per-question images in the first version, or is text-only acceptable for launch?

Summary

সংক্ষিপ্ত সারাংশ

In one paragraph

Four-level model: **Category** → **Exam** → **Question Set** → **Questions**, with questions in a reusable global bank that sets reference in order. Teachers submit papers as PDFs carrying a `#META` header, from which the parser reads duration, question count, marks, negative marking and sections. The backend parses asynchronously into a draft; an admin corrects anything the parser flagged on a review screen and publishes. Eleven phases, about eighteen working days, with the parser and review screen carrying the real risk.

বাংলায় সারাংশ

কাঠামো হবে চার স্তরের — **ক্যাটাগরি** → **পরীক্ষা** → **প্রশ্নসেট** → **প্রশ্ন**। একটি পরীক্ষার অধীনে অনেকগুলো প্রশ্নসেট থাকবে, যেগুলো শিক্ষকদের আপলোড করা বা সিস্টেমের তৈরি হতে পারে। প্রশ্নগুলো একটি সাধারণ প্রশ্নব্যাংকে থাকবে, আর প্রতিটি সেট সেখান থেকে ক্রমানুসারে প্রশ্ন রেফার করবে।

শিক্ষকরা একটি নির্দিষ্ট ফরম্যাটের PDF আপলোড করবেন। ফাইলের উপরে `#META` ব্লকে সময়, মোট প্রশ্নসংখ্যা, প্রতি প্রশ্নের নম্বর, নেগেটিভ মার্কিং এবং সেকশনের তালিকা লেখা থাকবে — সিস্টেম সেখান থেকেই সেটের সব কনফিগারেশন নেবে। তার নিচে `Q1.` · `A)` · `ANS:` · `EXP:` ফরম্যাটে প্রশ্নগুলো থাকবে।

ব্যাকএন্ড PDF থেকে লেখা বের করে প্রশ্নগুলো পার্স করবে এবং একটি ড্রাফট সেট তৈরি করবে। যেসব জায়গায় পার্সার নিশ্চিত নয়, সেগুলো আলাদাভাবে চিহ্নিত থাকবে। অ্যাডমিন একটি রিভিউ স্ক্রিনে সেগুলো ঠিক করে সেটটি পাবলিশ করবেন। ভুল থাকলে পাবলিশ করা যাবে না।

মোট **১১টি ধাপ**, আনুমানিক **১৮ কর্মদিবস**। দুজন ডেভেলপার একসাথে কাজ করলে প্রায় **১১ দিনে** শেষ করা সম্ভব। সবচেয়ে ঝুঁকিপূর্ণ অংশ দুটি — PDF পার্সার এবং রিভিউ স্ক্রিন।

Immediate next step

Two options, in order of usefulness:

1. **Lock the PDF format first.** Produce the `.docx` template and the sample PDF, circulate them to two or three teachers, and get real documents back before any parser code is written. This de-risks Phase 4 more than anything else on this list.
2. **Or start with Phase 0** — the home-page button and unlocking `/admin` — to get the panel reachable today and build outward from there.